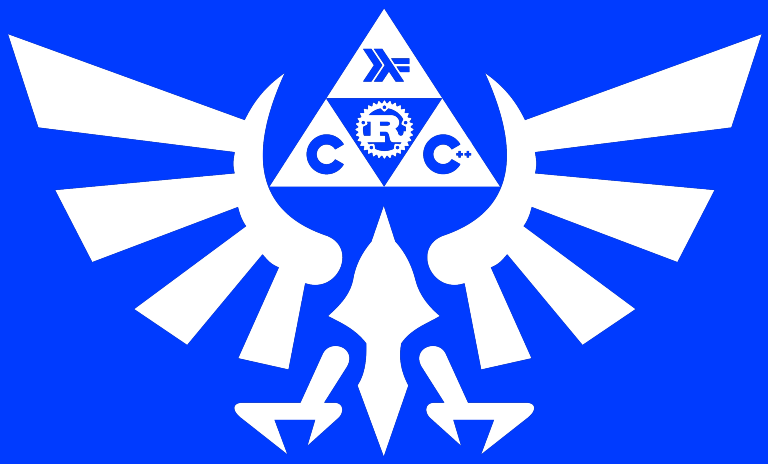
{EPITECH}

DAY 03_

< A GENTLE INTRODUCTION TO FUNCTIONAL PROGRAMMING />



DAY 03

binary name: Game.hs, Tree.hs, DoOp.hs, doop,
language: haskell
compilation: via Makefile, including re, clean and fclean rules

- ✓ The totality of your source files, except all useless files (binary, temp files, obj files,...), must be included in your delivery.
- ✓ All the bonus files (including a potential specific Makefile) should be in a directory named bonus.

Today's goal is to discover how to:

Part 1:
- handle errors in a more elegant way than with the error function.
- manage side effects (input and outputs).
- build an executable with Haskell.

Part 2:
- create custom types
- declare data structures
- handle algebraic data types
- create a binary tree
- implement a foldable type class

Good news: you have now the right to use most of the functions of the standard library. You are free to use functions from your My.hs if you want, but all of the useful functions are included (bug free) in the standard library in a way or another, just look for it (myMap is map, myTake is take, etc...).

Banned functions for today: fromJust, readMaybe, reads, elem, lookup.

You still have to respect the Haskell Coding Style.

All functions are going in DoOp.hs. Additionaly, a Makefile must be present so that when make is called in the delivery directory, a doop binary is built from DoOp.hs.

Part 1 - Step 0 - Prerequisite

Let's start with something simple.

Task 01

```haskell
myElem :: Eq a => a -> [a] -> Bool
```

A function which takes an argument which implements **equality** and a list of the same type, and returns True if the argument is present in the list, False otherwise.

For example:

```
Terminal
$>  ghci DoOp.hs
Prelude> myElem 3 [1,2,3,4,5]
True
Prelude> myElem 7 [1,2,3,4,5]
False
```

Step 1 - Maybe

Maybe is a data type included in the Haskell's standard library which is a simple and elegant way to handle errors without using exceptions. It's also a **monad**.

You can inspect it's definition with ghci:

```
Terminal
$>  ghci
Prelude> :info Maybe
data Maybe a = Nothing | Just a -- Defined in `GHC.Base'
[...]
```

Task 02

```
safeDiv :: Int -> Int -> Maybe Int
```

A function which takes two Ints as arguments, and returns **Nothing** if it's a division by zero, or **Just N** (with N the result of the division) if it's not.

Example:

```
Terminal
$>  ghci
Prelude> safeDiv 10 0
Nothing
Prelude> safeDiv 10 2
Just 5
```

Task 03

```
safeNth :: [a] -> Int -> Maybe a
```

A function similar to the function myNth, but which returns **Nothing** in case of an error, or **Just N** (with N the result) in case of success.

Task 04

```haskell
safeSucc :: Maybe Int -> Maybe Int
```

A function which takes a Maybe Int and returns its successor wrapped in a maybe.

```
Terminal
$>  ghci
Prelude> safeSucc (Just 41)
Just 42
Prelude> safeSucc (safeDiv 10 2)
Just 6
Prelude> safeSucc (safeDiv 10 0)
Nothing
```

Can you write safeSucc using the fmap function ?

Can you write safeSucc using the bind function/operator (»=) ?

Task 05

```haskell
myLookup :: Eq a => a -> [(a,b)] -> Maybe b
```

A function which takes an argument which implements equality and a list of tuples. Looks successively if the first element of each tuple is equal to the first argument. If a match is found, the second element of the tuple is returned wrapped in a **Maybe**. Otherwise, **Nothing** is returned.

Example:

```
Terminal
$>  ghci
Prelude> myLookup 2 [(1,"foo"),(2,"bar"),(3,"baz")]
Just "bar"
Prelude> myLookup 5 [(1,"foo"),(2,"bar"),(3,"baz")]
Nothing
```

Task 06

```haskell
maybeDo :: (a -> b -> c) -> Maybe a -> Maybe b -> Maybe c
```

A functions which takes a function as argument and two more arguments wrapped in **Maybes**, and returns Nothing if any of the arguments are **Nothing**, or **Just n** (with n the result of the function applied to the arguments) otherwise.

Example:

Terminal

```
$>  ghci
Prelude> maybeDo (\ x y -> x + y) (Just 1) (Just 2)
Just 3
Prelude> maybeDo (*) (safeDiv 10 0) (Just 5)
Nothing
Prelude> maybeDo (+) (safeDiv 10 5) (safeSucc $ Just 39)
Just 42
```

Can you write maybeDo using the bind function/opertor (»=) ?

Can you write maybeDo using the do notation?

Can you write maybeDo using fmap/(<$>) and the <*> operators?

Task 07

```
readInt :: [Char] -> Maybe Int
```

A function which takes a string as argument and returns Nothing if its not a parsable number, or an Int wrapped in a maybe otherwise.

Example:

```
Terminal
$>  ghci
Prelude> readInt "42"
Just 42
Prelude> readInt "foobar123"
Nothing
```

You're free to use the read function if you want, but **readMaybe** and **reads** are forbidden, obviously

Step 2 - IO

To handle Inputs and Outputs (IO), Haskell uses the IO monad, which permits to keep our function pure, while exchanging data from the outside world.

Task 08

```
getLineLength :: IO Int
```

A function which reads a line from the standard input and returns its length as a IO Int.

You should have a look on the getLine function

Task 09

```haskell
printAndGetLength :: String -> IO Int
```

A function which takes a string as argument, prints it on the standard output followed by a carriage return, and returns its length in the IO monad.

```
Terminal
$>  ghci
Prelude> printAndGetLength "hello world"
hello world
11
```

Task 10

```haskell
printBox :: Int -> IO ()
```

A function which takes an Int (N) as argument, and draw a box of height N and width N*2 on the standard output (see example bellow). If N is zero or less, it does nothing.

```
Terminal
$>  ghci
Prelude> printBox 5
+----------------+
|                |
|                |
|                |
+----------------+
Prelude>
```

Task 11

```
concatLines :: Int -> IO String
```

A function which takes an Int (N) as argument, read N lines from the standard input and returns a concatenation of these lines in the IO monad. If N is 0 or negative, an empty string is returned.

Example:

```
Terminal
$>  ghci
Prelude> concatLines 3
foo
bar
baz
"foobarbaz"
```

Task 12

```
getInt :: IO (Maybe Int)
```

A function which reads a line from the standard input and returns a Maybe Int in the IO monad.

Example:

```
Terminal
$>  ghci
Prelude> getInt
42
Just 42
```

Step 4 - doop

Task 13

A program called **doop**, which will be built by the command ”make”.

This program takes three arguments on the command line. The first and third arguments are numbers (signed integers). The second argument is an arithmetic operator (either + - * / or %). The program must print the result of the operation to the standard output.

In case of error, your program must return 84.

```
Terminal
$>  ./doop 1 + 2
3
$> ./doop 10 / 0
$> echo $?
84
```

Part 2 - Step 0 - Yet another RPG

You've been asked to implement the part of a role playing video-game handling hostile non-playing characters (Mobs).

Some of these mobs are able to hold an item in their hands. There are 3 different items and they are defined as:

- ✓ Sword
- ✓ Bow
- ✓ MagicWand

Task 14

Write the data structure to describe these items. The name of the data-structure must be Item, and each item can be created using its name:

Terminal

```
*Game> :t Sword
Sword :: Item
*Game> :t Bow
Bow :: Item
*Game> :i Item
[...]
```

Task 15

Item must be deriving from **Eq**, so you should be able to compare two items.

Terminal

```
*Game> Sword == Bow
False
*Game> Sword == Sword
True
*Game> Sword /= MagicWand
True
```

Task 16

Now we want items to be displayed in a specific manner when we print them. To achieve this we want **Item** to be an instance of **Show** . Provide a custom implementation to achieve this behavior:

Terminal

```
*Game> Sword
sword
*Game> Bow
bow
*Game> MagicWand
magic wand
```

Step 1 - the Mobs

Now that we have well defined items, we're ready to implement the mobs.

Task 17

There are 3 kinds of mobs in this game: **Mummy**, **Skeleton** and **Witch**

- ✓ All the mummies come bare handed.
- ✓ The skeleton always holds an item.
- ✓ A witch can either **Just** hold an item or **Nothing**.

Create a data structure to represent them.

There's not a lot of ways to do that properly...

Your data structure **Mob** must be deriving from **Eq** and **Show**

Task 18

But some also come in different varieties, therefore you must implement the following functions to create each specific type of mob:

```
createMummy :: Mob      -- a Mummy
createArcher :: Mob     -- a Skeleton holding a Bow
createKnight :: Mob     -- a Skeleton holding a Sword
createWitch :: Mob      -- a Witch holding Nothing
createSorceress :: Mob -- a Witch holding a MagicWand
```

Task 19

We also want a single function to create any kind of mob depending on a string. As this function can fail if the string doesn't correspond to any mob, it returns a Maybe Mob.

```
create :: String -> Maybe Mob
```

The string coresponding to each type of mob are respectively:

- ✓ ”mummy”
- ✓ ”doomed archer”
- ✓ ”dead knight”
- ✓ ”witch”
- ✓ ”sorceress”

Task 20

We also want to be able to equip Skeletons and Witch with any Item, replacing the one they already hold. As mummies can't hold anything, this function can also fail...

```
equip :: Item -> Mob -> Maybe Mob
```

Task 21

As with items, we want to provide a custom way to print mobs.

If you provide a custom implementation you have to remove **Show** from the list of type classes your data type ”automaticaly” derives from...

- ✓ a Mummy must be shown as ”mummy”
- ✓ a Skeleton holding a Bow must be shown as ”doomed archer”
- ✓ a Skeleton holding a Sword must be shown as ”dead knight”
- ✓ a Skeleton holding anything else as ”skeleton holding a ” followed by the said item
- ✓ a Witch holding nothing must be shown as ”witch”
- ✓ a Witch holding a MagicWand must be shown as ”sorceress”
- ✓ and a Witch holding another item as ”witch holding a ” followed by the said item.

Task 22

Now it is time to create our own type class. We want to have a way to know if a Mob, NPCs, Player, or any other character in the game currently holds an Item or not.

To achieve this you have to declare a new type class called **HasItem**.

This type class has two functions:

- ✓ getItem which takes an object as argument and returns a Maybe Item.
- ✓ hasItem which takes an object as argument and returns a Bool.

The minimal definition for this class is to provide an implementation for getItem, a generic hasItem implementation must be provided by the type class definition.

Task 23

Make **Mob** an instance of **HasItem**

Step 2 - Binary Tree

In this last part, we're going to do something quite diffrent and more generic: you're going to create you own ”container” data structure which will be a simple binary tree.

For these exercices, create a new source file called **Tree.hs**

Task 24

Define a new data type called **Tree** which takes a type **”a”** :

```
data Tree a = ...
```

A Tree can either be constructed with **Empty** as a constructor, or with the constructor **Node** which contains, in order, another Tree of the same type, a value of type **a**, and a last Tree. your Tree must be an instance of **Show**

At this point you should be able to create trees *”by hand”* such as:

```
Terminal
> Empty
Empty
> Node Empty 42 Empty
Node Empty 42 Empty
> Node (Node Empty 31 Empty) 42 (Node Empty 53 Empty)
Node (Node Empty 31 Empty) 42 (Node Empty 53 Empty)
```

You should have a look on the definition of **Maybe** or lists...

Task 25

We now want a simple function to add a new value value in an existing tree. To add a new value in the right place in our tree, we need to be able to compare the values between each other, so for this function **"a"** must be an instance of **Ord**.

```
addInTree :: Ord a => a -> Tree a -> Tree a
```

This function takes a value, a tree, and returns a new tree with this value added in the right spot.

The rules to add a new value are as follow:

- ✓ If the tree is **Empty**, create a **Node** containing the value, with **Empty** as its right and left branch.
- ✓ If the tree is a **Node** :
  - – if the value to be inserted is striclty lower than the value stored in this **Node**, the value must be added to its left branch.
  - – if it's greater or equal, the value must be added to its right branch.

Example:

```
Terminal
> addInTree 42 Empty
Node Empty 42 Empty
> addInTree 31 $ addInTree 42 Empty
Node (Node Empty 31 Empty) 42 Empty
> addInTree 53 $ addInTree 31 $ addInTree 42 Empty
Node (Node Empty 31 Empty) 42 (Node Empty 53 Empty)
```

Task 26

We want to be able to apply a function on all the values stored in our **Tree**. To achieve this, make your tree an instance of the **Functor** type class.

```
Terminal
> fmap (*2) $ addInTree 53 $ addInTree 31 $ addInTree 42 Empty
Node (Node Empty 62 Empty) 84 (Node Empty 106 Empty)
```

Task 27

We want to be able to convert a list of values to a **Tree**. Write the function **listToTree** which takes a list of **”a”** as argument and returns a Tree build using addInTree.

Task 28

Likewise, we want to be able to concert a **Tree** to a list. Write a function **treeToList** which takes a tree of **”a”** as argument and returns a **[a]**.

Task 29

Using your existing function, write the function treeSort, which takes a list of **”a”** and returns a sorted list of **”a”**.

Task 30

Make your **Tree** an instance of **Foldable**.

It should look like generalised version of a function you have alread written

Once you've done it, you should be able to re-write some functions in a shorter manner

v 2.1